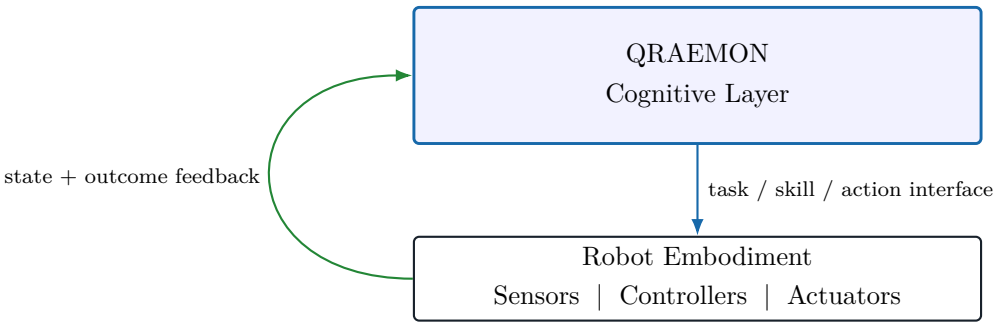
QRAEMON

Robotics Cognitive Architecture

Technical Architecture & Product Specification



Audience:	CTO, Robotics Engineers, AI/ML Engineers, Controls Engineers, Systems Engineers, Embedded Engineers, Technical Architects
Scope:	Two deployable product forms (1) Software-only cognitive layer (2) Fully offline embedded cognitive module
Status:	Technical architecture / engineering specification
Version:	1.0

This document defines an engineering architecture derived from the QRAEMON robotics integration proposal and the product concept developed for integration with existing robots.

Contents

1 Executive Technical Summary **3**

2 Product Definitions **3**
2.1 Product A — QRAEMON Software . . . 3
2.2 Product B — QRAEMON Embedded . . . 3
2.3 Shared Product Principle . . . 4

3 Baseline: Existing Robot Control Architecture **4**
3.1 Before QRAEMON — Canonical Existing-Robot Diagram . . . 4

4 After QRAEMON — Correct Integration Architecture **5**
4.1 Canonical Post-QRAEMON Block Diagram . . . 5

5 Architectural Boundary **5**
5.1 Responsibility Allocation . . . 5

6 QRAEMON Cognitive Runtime **6**
6.1 High-Level Runtime . . . 6
6.2 Core Modules . . . 6
6.2.1 1. State and Context Manager . . . 6
6.2.2 2. Experience Memory . . . 6
6.2.3 3. Reasoning Engine . . . 7
6.2.4 4. Planner . . . 7
6.2.5 5. Action/Skill Manager . . . 8

7 Canonical Robot Interface **8**
7.1 Design Objective . . . 8
7.2 Robot State Interface . . . 8
7.3 Action Interface . . . 9
7.4 Adapter Responsibilities . . . 9

8 Product A — Software-Only Architecture **10**
8.1 Deployment Model . . . 10
8.2 Advantages . . . 10
8.3 Engineering Requirements . . . 10

9 Product B — Fully Offline Embedded Architecture **10**
9.1 Product Definition . . . 10
9.2 Embedded Hardware Architecture . . . 11
9.3 Robot Integration . . . 11
9.4 Offline-First Requirements . . . 11

10 Communication and Data Flow **12**
10.1 Three Logical Data Planes . . . 12
10.2 Timing Classes . . . 12

11 Safety Architecture **12**
11.1 Safety Hierarchy . . . 13
11.2 Action Guard . . . 13

12 Memory and Continual Learning **13**
12.1 Experience Lifecycle . . . 13

13 Robot Adapter SDK **14**
13.1 Why the Adapter Exists . . . 14
13.2 Adapter Contract . . . 14
13.3 Canonical Units and Frames . . . 14

14 Software Architecture **15**
14.1 Process Isolation . . . 16

15 Failure Handling **16**
15.1 Examples . . . 16

16 Security Architecture **16**

17 Observability and Diagnostics **17**

18 Development and Validation Roadmap **17**
18.1 Engineering Phase 0 — Interface First . . . 17
18.2 Phase 1 — Simulation . . . 18
18.3 Phase 2 — Physical MVP . . . 18
18.4 Phase 3 — Memory Proof . . . 18

19 Testing Strategy **19**
19.1 Unit Tests . . . 19
19.2 Integration Tests . . . 19
19.3 Hardware-in-the-Loop . . . 19
19.4 Memory Ablations . . . 19

20 Software vs. Embedded Product Comparison **20**

21 What QRAEMON Is Not **20**

22 CTO-Level Design Principles **21**

23 Reference End-to-End Architecture **21**

24 Implementation Acceptance Criteria **21**

25 Recommended Immediate Engineering Backlog **22**

26 Final Architectural Position **22**

1 Executive Technical Summary

QRAEMON is specified here as a **robot-agnostic cognitive layer**: a software architecture that can perceive relevant robot/environment state, maintain and retrieve experience, reason over the current context, select strategies or skills, issue bounded high-level actions, evaluate outcomes, and improve future decisions through accumulated experience.

The existing robotics control stack remains responsible for deterministic physical control, hard safety constraints, collision limits, emergency stop behavior, and actuator-level execution. This separation is consistent with the supplied QRAEMON proposal, which explicitly assigns task/skill-level cognition to QRAEMON and retains motion planning, control, and safety authority in the existing robotics stack.

> **Core architectural rule.** QRAEMON shall **not initially replace low-level motor controllers or safety-critical controllers**. It shall operate above the physical control layer through a defined task/skill interface. The robot-specific adapter translates between QRAEMON's canonical interface and the interfaces exposed by the target robot.

Two products are defined:

1. QRAEMON Software: a software-only runtime that can run on an existing robot computer, workstation, edge server, or other suitable compute platform.
2. QRAEMON Embedded: a self-contained embedded/edge computer installed *inside* a robot, running QRAEMON locally and requiring no external real-time connection or cloud service.

The two products share the same cognitive core and canonical robot interface. The primary difference is deployment and hardware ownership, not cognitive architecture.

2 Product Definitions

2.1 Product A — QRAEMON Software

QRAEMON Software is a deployable software runtime.

Purpose: add a cognitive layer to an existing robot without requiring a dedicated QRAEMON hardware computer.

Typical deployment:

- Existing robot industrial PC.
- Existing robot controller computer, where compute isolation is acceptable.
- External workstation/edge computer connected through a permitted robot interface.
- Simulation environment for development and validation.

The software exposes a stable API and consumes robot state/perception data through a robot-specific adapter. It returns bounded task/skill commands to the existing robot stack.

2.2 Product B — QRAEMON Embedded

QRAEMON Embedded is the same cognitive architecture packaged with dedicated onboard compute, storage, I/O and security hardware.

Its design objective is:

> *Install the cognitive computer inside the robot so that the robot can perceive, reason, plan and act locally without an Internet connection, cloud dependency, or external real-time computer.*

The module still requires internal electrical/data interfaces to the robot's sensors and control infrastructure. "Offline" means no external network or external real-time cognitive computer is required; it does not mean that the internal nervous-system-like data pathways are absent.

2.3 Shared Product Principle

The following components should be shared as much as possible:

- Cognitive runtime.
- Memory architecture.
- Experience representation.
- Reasoning/planning APIs.
- Canonical robot state schema.
- Canonical action/skill schema.
- Safety policy interface.
- Robot adapter SDK.
- Evaluation and observability tooling.

The hardware-specific components belong primarily to QRAEMON Embedded.

3 Baseline: Existing Robot Control Architecture

A conventional robot can be abstracted as a closed-loop control system. The low-level controller receives desired state/setpoints and measured internal state, computes control commands, and drives actuators. The physical robot produces new state, which is measured again.

External/environment sensors may provide perception information to a higher-level computer or planner; they are not required to be part of the low-level feedback loop.

3.1 Before QRAEMON — Canonical Existing-Robot Diagram

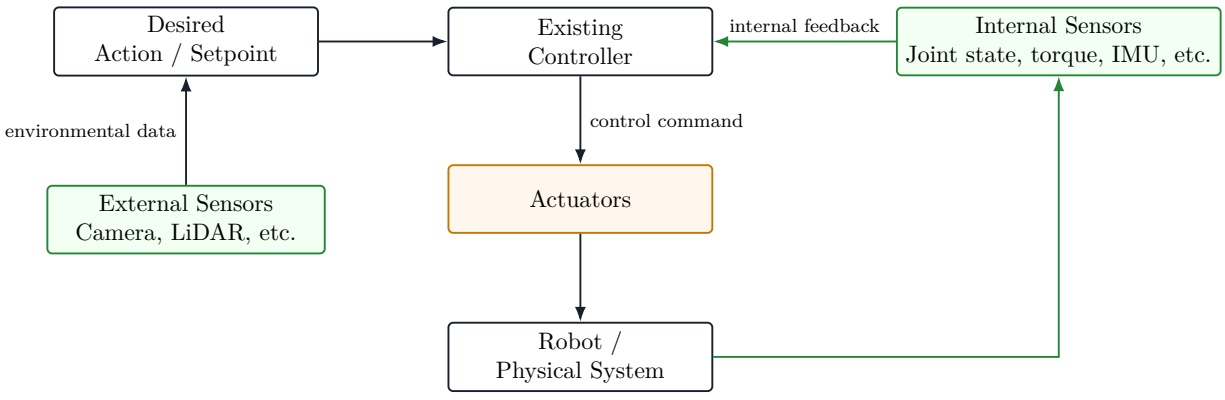


Figure 1: Canonical abstraction of an existing robot system before QRAEMON.

Important: the exact location of external perception processing varies by robot. Figure 1 intentionally abstracts it as information available to a higher-level system rather than asserting a specific manufacturer architecture.

4 After QRAEMON — Correct Integration Architecture

QRAEMON is introduced as a cognitive layer above the existing physical control system. It receives the information needed to understand the current state and environment, uses memory and reasoning to select a strategy or skill, and sends a bounded high-level action to the existing robot execution stack.

4.1 Canonical Post-QRAEMON Block Diagram

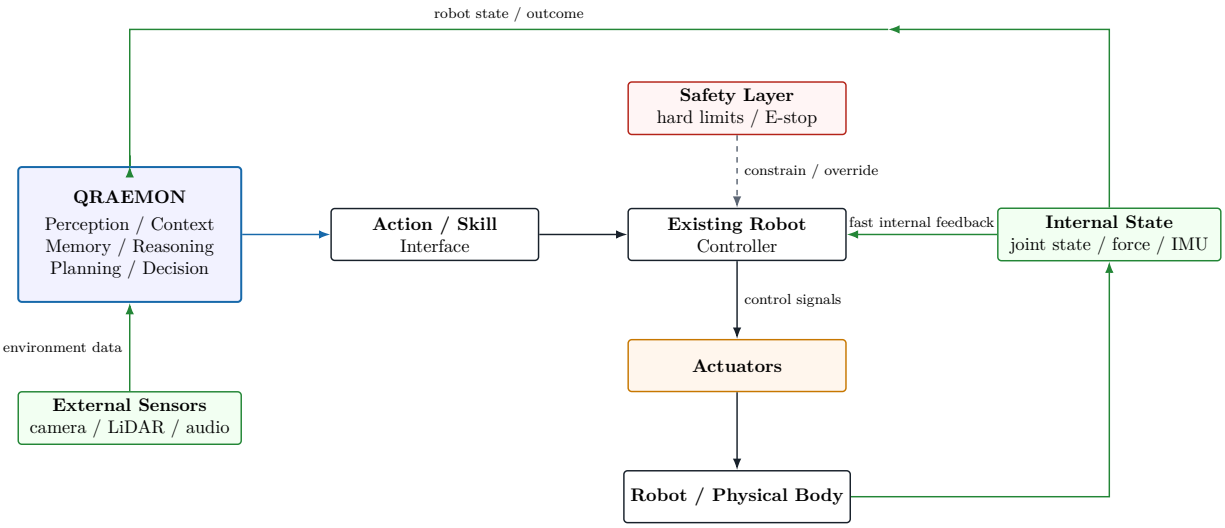


Figure 2: Canonical QRAEMON-integrated robot architecture. QRAEMON adds a cognitive loop without removing the existing low-level control loop.

Control-loop preservation. The existing internal sensor → controller feedback path remains intact. QRAEMON does not replace that fast closed-loop path. QRAEMON receives appropriate robot-state/outcome information and sends higher-level actions through an explicit interface.

5 Architectural Boundary

5.1 Responsibility Allocation

Layer	QRAEMON responsibility	Existing robotics responsibility
Perception/state	Interpret current context; fuse relevant state available through interfaces	Sensors, state estimation, low-level perception where already present
Memory	Retrieve, evaluate, consolidate and decay experience	Optional logging/data infrastructure
Reasoning	Determine strategy under context and uncertainty	Deterministic execution constraints
Planning	Select/sequence skills at cognitive/task level	Motion planning and trajectory generation where applicable
Action	Issue bounded task/skill-level commands	Translate into robot-specific execution

Control	No ownership of low-level servo loop in initial architecture	Position/velocity/torque control
Safety	Policy-level constraints and decision gating	Hard limits, emergency stop, collision protection, safety PLC/controller
Feedback	Evaluate outcomes and store useful experience	Telemetry and sensor feedback

This boundary follows the supplied proposal's product architecture and scope discipline: QRAEMON operates at the cognitive/task/skill level while deterministic planners, controllers and safety systems retain physical-actuation authority.

6 QRAEMON Cognitive Runtime

6.1 High-Level Runtime

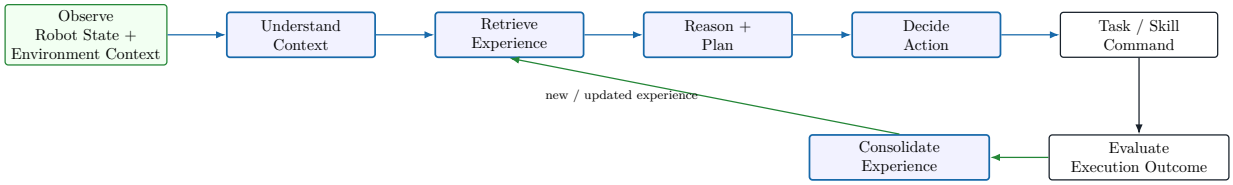


Figure 3: Cognitive runtime loop.

The intended conceptual loop is:

$$\boxed{\text{Observe} \rightarrow \text{Remember} \rightarrow \text{Reason} \rightarrow \text{Act} \rightarrow \text{Learn} \rightarrow \text{Evolve}}$$

The supplied proposal identifies this loop and places selective experience memory at the center of the first product thesis.

6.2 Core Modules

6.2.1 1. State and Context Manager

Responsibilities:

- Maintain the current robot state.
- Maintain task state.
- Maintain environmental/object context.
- Timestamp and version state.
- Track uncertainty and data freshness.
- Normalize robot-specific data into a canonical representation.

6.2.2 2. Experience Memory

Memory is not merely a sensor-data archive. The proposed initial experience representation includes:

Field	Purpose
Context	Task, environment, objects, state and constraints
Goal	What the robot was trying to achieve
Action	Selected skill/strategy and relevant parameters
Outcome	Success, failure, partial result and consequence
Cause/explanation	Why the outcome occurred, where inferable
Utility	Estimated future value, reliability and surprise
Associations	Links to related tasks, objects, actions and episodes
Temporal signals	Recency, repetition, decay and consolidation state

Table 2: Initial experience representation.

Required memory operations:

- Store.
- Retrieve.
- Rank.
- Associate.
- Compress.
- Consolidate.
- Decay/forget.
- Detect contradiction.
- Evaluate usefulness.

6.2.3 3. Reasoning Engine

The reasoning engine consumes:

$$(\text{current context}, \text{goal}, \text{retrieved experience}, \text{constraints})$$

and produces candidate strategies, hypotheses, or skill sequences.
It must expose:

- Confidence.
- Preconditions.
- Expected outcome.
- Relevant memory references.
- Resource requirements.
- Safety constraints.
- Recovery/fallback strategy.

6.2.4 4. Planner

The planner converts cognitive decisions into an executable sequence of skills.
Example:

```
Goal: Pick up red cup.

1. locate(red_cup)
2. move_to(pregrasp_pose)
3. orient(gripper)
4. grasp(red_cup, force_limit)
5. verify(grasp)
6. lift(height)
```

The planner should not generate raw motor currents in the initial architecture.

6.2.5 5. Action/Skill Manager

Canonical actions should be capability-oriented rather than hardware-oriented.

Examples:

- MOVE_TO
- GRASP
- RELEASE
- PICK
- PLACE
- FOLLOW
- NAVIGATE
- INSPECT
- WAIT
- RECOVER

Robot-specific execution is delegated through the adapter.

7 Canonical Robot Interface

The most important reusable engineering asset is the interface between QRAEMON and a robot.

7.1 Design Objective

A new robot should require a new **adapter**, not a rewrite of the QRAEMON cognitive core.

$$\boxed{\text{QRAEMON Core} + \text{Robot Adapter} = \text{Robot Deployment}}$$

7.2 Robot State Interface

A canonical state packet should support fields such as:

```
{
  "timestamp": 1729160000.123,
  "robot_id": "robot_001",
  "joints": {
    "position": [],
    "velocity": [],
    "effort": []
  },
```

```
  "pose": {
    "position": [0.0, 0.0, 0.0],
    "orientation": [1.0, 0.0, 0.0, 0.0]
  },
  "forces": {},
  "imu": {},
  "battery": {},
  "task_state": {},
  "events": [],
  "capabilities": [],
  "environment": {}
}
```

The actual schema should be versioned and extensible.

7.3 Action Interface

A canonical action request can be represented as:

```
{
  "action_id": "a-001",
  "type": "GRASP",
  "target": {
    "object_id": "red_cup"
  },
  "parameters": {
    "force_limit": 12.0
  },
  "constraints": {
    "max_duration_s": 5.0,
    "collision_policy": "strict"
  },
  "confidence": 0.91
}
```

The adapter maps this to the target robot's execution mechanism.

7.4 Adapter Responsibilities

An adapter should implement:

1. `connect()` / `disconnect()`.
2. `get_state()`.
3. `get_capabilities()`.
4. `execute_skill()`.
5. `cancel_action()`.
6. `get_action_status()`.
7. `get_events()`.
8. `health_check()`.
9. Optional perception/state-estimation hooks.

The adapter is not the cognitive brain. It is the embodiment translation layer.

8 Product A — Software-Only Architecture

8.1 Deployment Model

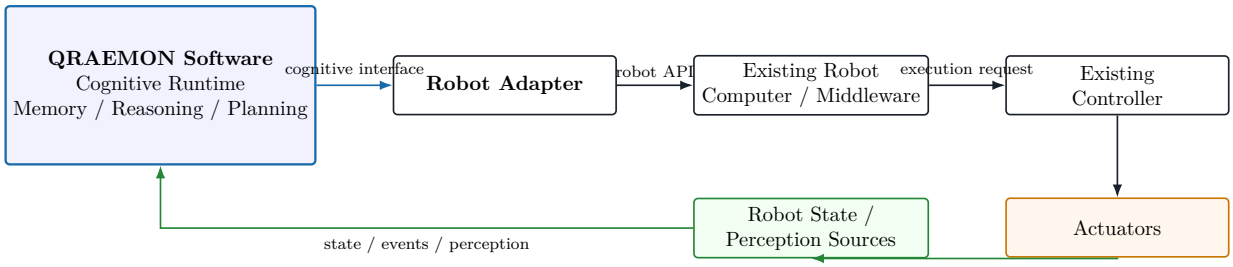


Figure 4: Software-only QRAEMON deployment.

8.2 Advantages

- No dedicated QRAEMON hardware required.
- Fastest route to simulation and software validation.
- Suitable for research labs and existing robot PCs.
- Same cognitive core can later be moved to embedded hardware.
- Easier debugging and development.

8.3 Engineering Requirements

- Linux-first runtime is recommended for the initial implementation.
- Containerized deployment should be supported where compatible with real-time requirements.
- The cognitive runtime must be isolated from safety-critical control processes.
- Communication must support timeouts, retries, stale-data detection and action cancellation.
- Every action must have an explicit lifecycle: `PROPOSED` → `AUTHORIZED` → `EXECUTING` → `SUCCEEDED`/`FAILED`/`CANCELLED`.

9 Product B — Fully Offline Embedded Architecture

9.1 Product Definition

QRAEMON Embedded is a dedicated computing module physically installed inside the robot. It requires:

- Electrical power from the robot.
- Internal data interfaces to the robot's sensors/state and execution infrastructure.

It does *not* require:

- Internet access.
- Cloud access.
- External real-time compute.
- An external QRAEMON server.
- Continuous remote operator connection.

9.2 Embedded Hardware Architecture

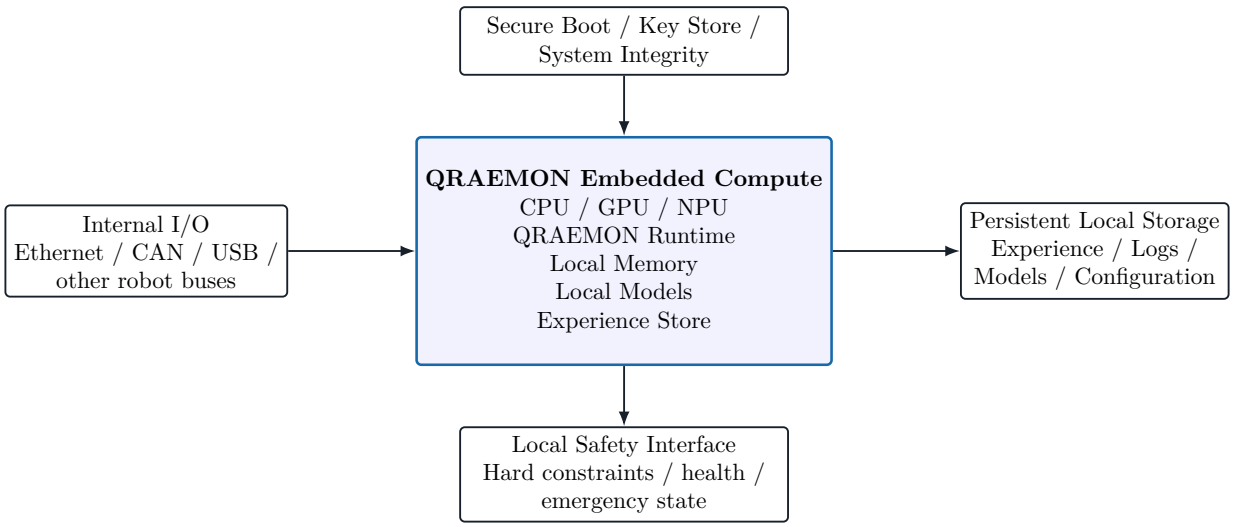


Figure 5: Conceptual QRAEMON Embedded module.

9.3 Robot Integration

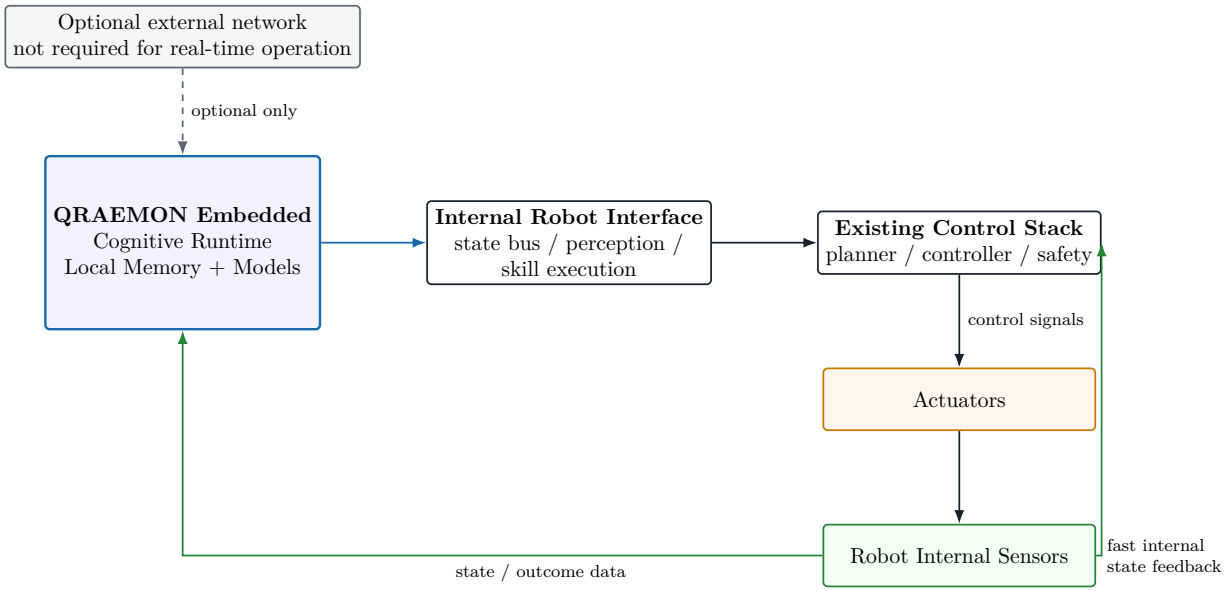


Figure 6: Fully offline QRAEMON deployment inside a robot. Internal robot state feeds the existing control stack's fast feedback loop while QRAEMON consumes the same state and outcome information through the internal interface.

9.4 Offline-First Requirements

The embedded product shall satisfy:

1. Boot and operate without Internet access.
2. Load all mandatory models and runtime dependencies locally.
3. Maintain local experience memory.
4. Continue the cognitive loop when external network access is unavailable.

5. Detect stale or missing sensor/state data.
6. Fail safely when cognitive output is unavailable.
7. Never bypass hard safety controls.
8. Support local diagnostics and logs.
9. Support signed offline software/model updates.

10 Communication and Data Flow

10.1 Three Logical Data Planes

A robust architecture should separate:

1. **Cognitive plane**: context, memory, reasoning, planning, task/skill decisions.
2. **Control plane**: trajectories, servo commands, state feedback and execution status.
3. **Safety plane**: emergency stop, hard limits, collision constraints and safety interlocks.

QRAEMON owns the cognitive plane. It interfaces with the control plane but should not own the safety plane in the initial product.

10.2 Timing Classes

Not every signal requires the same latency.

Class	Typical character	Architectural treatment
Safety	Hard real-time / immediate	Independent safety path; never depend on cognitive inference
Servo control	High-rate deterministic	Existing controller retains authority
State feedback	High-rate / periodic	QRAEMON consumes appropriately sampled state
Perception	Medium/high latency	Timestamped and uncertainty-aware
Cognition	Task-dependent	Explicit latency budget; asynchronous where appropriate
Memory consolidation	Background	Can run opportunistically without blocking control

Table 3: Logical timing classes. Exact rates are robot-specific and must be specified during integration.

11 Safety Architecture

Safety is a hard architectural boundary.

> **QRAEMON must never be the sole safety mechanism.** A cognitive model may be uncertain, wrong, unavailable, corrupted, delayed, or operating on stale information. Existing robot safety mechanisms must remain capable of constraining or stopping physical motion independently of QRAEMON.

11.1 Safety Hierarchy

1. Emergency stop / hard safety mechanisms.
2. Hardware/firmware limits.
3. Safety controller / safety PLC where applicable.
4. Existing low-level robot controller.
5. Motion planner and execution constraints.
6. QRAEMON action authorization and cognitive policy.

QRAEMON therefore proposes actions; the robot system decides whether they are physically permissible.

11.2 Action Guard

Every action should pass through a guard:

$$A_{\text{requested}} \rightarrow \text{Schema Validation} \rightarrow \text{Capability Check} \rightarrow \text{Constraint Check} \rightarrow \text{Safety Authorization} \rightarrow A_{\text{executed}}$$

A failed guard must not produce uncontrolled actuator output.

12 Memory and Continual Learning

The central research thesis in the supplied proposal is that a robot that repeatedly encounters related situations should use useful prior experience rather than repeatedly solving the situation from zero.

The first product should therefore test whether selective memory improves:

- Task success rate.
- Recovery success.
- Sample efficiency.
- Retrieval precision.
- Transfer to related conditions.
- Stability of previously learned capabilities.
- Bounded memory growth.
- End-to-end latency.

12.1 Experience Lifecycle

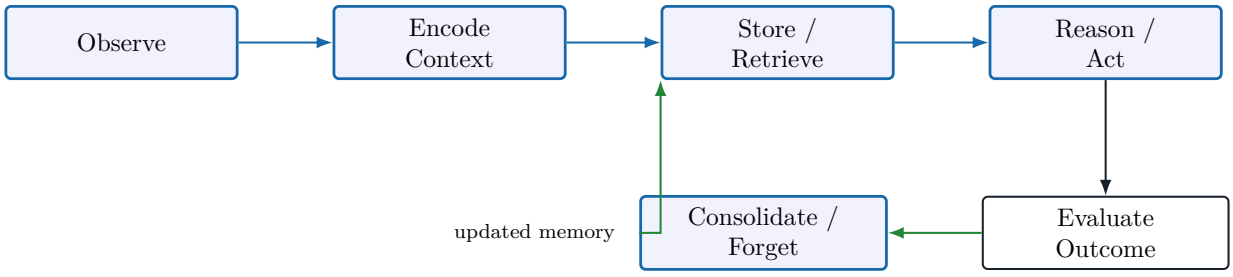


Figure 7: Experience as an evolving computational process.

Memory should not become an uncontrolled sensor-data warehouse. Storage, retrieval and consolidation require explicit budgets and policies.

13 Robot Adapter SDK

13.1 Why the Adapter Exists

The adapter isolates embodiment-specific engineering.

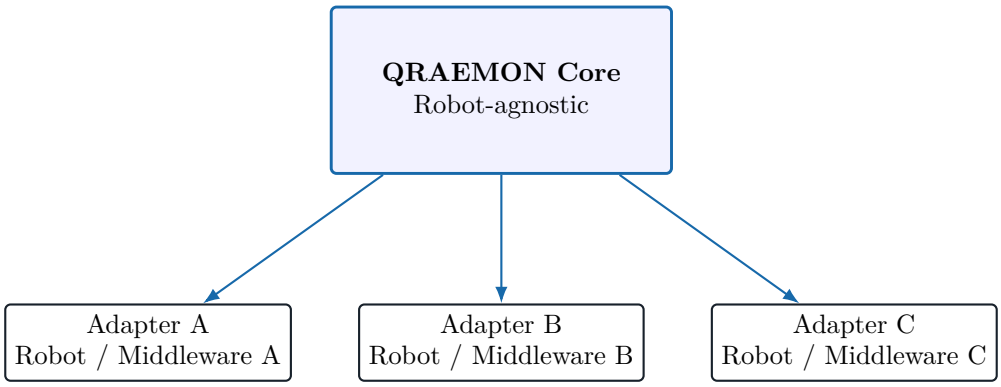


Figure 8: One cognitive core, multiple robot adapters.

13.2 Adapter Contract

A production adapter must specify:

- Supported robot capabilities.
- Supported action types.
- State update semantics.
- Timestamp and clock source.
- Coordinate frames.
- Units.
- Action cancellation behavior.
- Error semantics.
- Timeout behavior.
- Safety-state behavior.
- Authentication/authorization if applicable.
- Version compatibility.

13.3 Canonical Units and Frames

The QRAEMON interface should use explicit SI units and explicit coordinate-frame identifiers. Never rely on undocumented conventions.

Examples:

- Position: metres.
- Velocity: metres/second.

- Angular position: radians.
- Angular velocity: radians/second.
- Force: newtons.
- Torque: newton-metres.
- Time: seconds, with monotonic timestamps for runtime scheduling.

14 Software Architecture

A recommended initial repository structure is:

```
qraemon/
|
+-- core/
|   +-- context/
|   +-- perception/
|   +-- memory/
|   +-- reasoning/
|   +-- planning/
|   +-- decision/
|   +-- learning/
|
+-- robot/
|   +-- canonical_state/
|   +-- action_api/
|   +-- capability_api/
|   +-- adapter_sdk/
|   +-- adapters/
|
+-- safety/
|   +-- policy/
|   +-- validation/
|   +-- authorization/
|
+-- runtime/
|   +-- scheduler/
|   +-- event_bus/
|   +-- telemetry/
|   +-- health/
|
+-- storage/
|   +-- experience/
|   +-- configuration/
|   +-- model_registry/
|
+-- simulation/
|
+-- tools/
|
+-- tests/
|
```

```
├── deployment/
│   ├── software/
│   ├── embedded/
│
├── docs/
```

14.1 Process Isolation

A recommended initial process boundary is:

- `qraemon-runtime`: cognitive orchestration.
- `state-service`: canonical state and synchronization.
- `memory-service`: persistent experience.
- `adapter-service`: robot communication.
- `safety-gateway`: action validation/authorization.
- `telemetry-service`: logs and observability.

The exact process topology may be collapsed for a small embedded target, but the logical interfaces should remain intact.

15 Failure Handling

QRAEMON must treat uncertainty and failure as first-class states.

15.1 Examples

Failure	**QRAEMON response**	**Robot-side requirement**
Sensor timeout	Mark state stale; stop or replan as policy dictates	Controller/safety remains operational
Adapter disconnected	Cancel/hold cognitive actions	Robot enters defined safe state
Low confidence	Reject, defer or request additional perception	No uncontrolled execution
Planner failure	Invoke recovery strategy or fail safely	Execution cancellation supported
Memory unavailable	Continue degraded mode if safe	Cognitive functions degrade gracefully
Model failure	Fall back to deterministic behavior or safe state	Existing controller remains authoritative
Network loss	Continue local operation in offline modes	No cloud dependency for core loop
Conflicting memories	Reduce confidence; require stronger evidence	No automatic unsafe action

16 Security Architecture

For QRAEMON Embedded, security must be designed for disconnected operation.

- Secure boot.
- Signed software packages.

- Signed model packages.
- Encrypted persistent storage where required.
- Device identity.
- Role-based local authorization.
- Immutable or tamper-evident security logs where required.
- Explicit update/recovery procedure.
- No implicit trust in network availability.

Offline operation does not remove the need for cybersecurity; it changes the threat model.

17 Observability and Diagnostics

Every action should produce a traceable record containing at least:

- Input context version.
- Relevant memory IDs.
- Reasoning/planning version.
- Selected action.
- Confidence.
- Safety decision.
- Execution result.
- Outcome metrics.
- Timing/latency.

This enables causal evaluation of the central QRAEMON claim: whether experience actually changes future decisions and improves outcomes.

18 Development and Validation Roadmap

The supplied proposal recommends a staged path:

1. Architecture.
2. Simulation.
3. Physical MVP.
4. Experience proof.
5. External pilot.
6. Productization.

18.1 Engineering Phase 0 — Interface First

Deliver:

- Canonical state schema.
- Canonical action/skill schema.

- Adapter SDK.
- Capability model.
- Action lifecycle.
- Safety gateway interface.
- Experience schema.

18.2 Phase 1 — Simulation

Build:

- Simulated robot adapter.
- Deterministic benchmark tasks.
- Sensor/state stream.
- Existing-controller simulation.
- QRAEMON cognitive loop.
- No-memory baseline.
- Memory-enabled experiment.

18.3 Phase 2 — Physical MVP

The supplied proposal recommends controlled robotic manipulation using a 6-DOF collaborative/research manipulator, RGB-D plus robot state, with optional force/torque sensing.

Initial tasks can include:

- Pick-and-place.
- Sorting.
- Insertion.
- Tool use.
- Failure recovery.

18.4 Phase 3 — Memory Proof

The experiment should compare:

$$\text{Baseline Robot} \quad \text{vs.} \quad \text{Robot + QRAEMON Memory}$$

under matched conditions.

Measure:

$$\Delta\text{Success Rate}, \quad \Delta\text{Recovery Rate}, \quad \Delta\text{Sample Efficiency}, \quad \Delta\text{Transfer}$$

and verify that gains are causal rather than the result of additional uncontrolled compute or unmatched training.

19 Testing Strategy

19.1 Unit Tests

Test:

- State normalization.
- Coordinate conversions.
- Action schema validation.
- Memory retrieval.
- Memory consolidation.
- Planner outputs.
- Safety guards.
- Adapter error handling.

19.2 Integration Tests

Test:

- State → QRAEMON.
- QRAEMON → adapter.
- Adapter → existing robot stack.
- Action completion/failure reporting.
- Sensor dropout.
- Controller dropout.
- Cancellation.
- Recovery.

19.3 Hardware-in-the-Loop

The first hardware tests should constrain:

- Maximum velocity.
- Maximum acceleration.
- Workspace.
- Force/torque.
- Energy.
- Allowed action set.

19.4 Memory Ablations

At minimum:

1. No memory.
2. Raw retrieval baseline.

3. QRAEMON selective memory.
4. QRAEMON memory without consolidation.
5. QRAEMON memory without retrieval ranking.

The purpose is to identify which mechanism causes the measured improvement.

20 Software vs. Embedded Product Comparison

Dimension	QRAEMON Software	QRAEMON Embedded
Core cognition	Same	Same
Deployment	Existing computer / permitted host	Dedicated onboard computer
External real-time connection	May use one depending on integration	Not required
Cloud dependency	Optional	Optional, not required
Robot adapter	Required	Required
Local memory	Yes	Yes
Local inference	Yes if hardware permits	Yes by design
Offline operation	Possible	Primary requirement
Hardware ownership	Robot/customer	QRAEMON product
Portability	High	High with hardware integration
Primary early use	Research/software integration	Autonomous/secure/offline deployment

Table 5: Product comparison.

21 What QRAEMON Is Not

For technical clarity, the initial product should not be described as:

- A replacement for a servo controller.
- A replacement for an emergency-stop system.
- A raw motor-current generator.
- A cloud-only robot brain.
- A giant sensor-data database.
- A claim of consciousness or human equivalence.
- A requirement to retrain a giant model after every experience.

Instead:

> **QRAEMON is a persistent cognitive layer that uses current context and accumulated experience to improve task/skill-level decisions while existing robotics control and safety systems retain authority over physical actuation.**

This wording is aligned with the supplied proposal's product definition and scope.

22 CTO-Level Design Principles

1. **Cognition must be embodiment-agnostic.**
2. **Interfaces must be stable before models become complex.**
3. **Safety must remain independent of cognitive inference.**
4. **Offline operation must be a supported architecture, not a retrofit.**
5. **Memory must be selective, bounded and evaluable.**
6. **Every cognitive action must be observable.**
7. **Every robot integration must be adapter-based.**
8. **Simulation must precede unconstrained physical deployment.**
9. **Claims of learning must be demonstrated through ablation and repeated trials.**
10. **The same QRAEMON core should survive changes in robot embodiment.**

23 Reference End-to-End Architecture

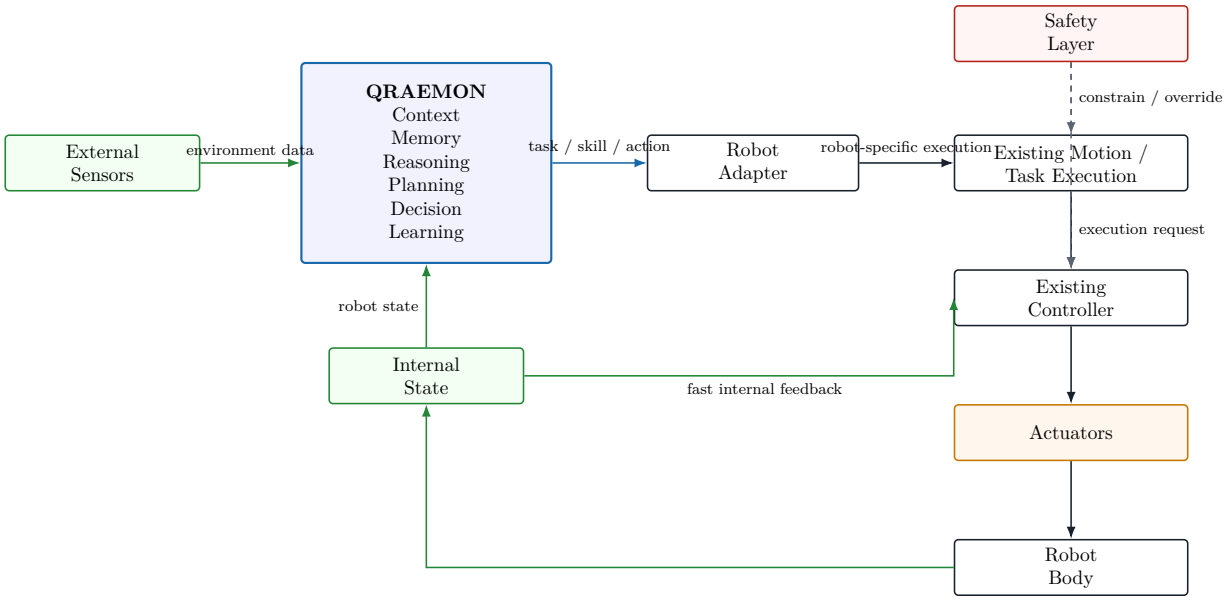


Figure 9: Reference end-to-end QRAEMON architecture.

24 Implementation Acceptance Criteria

A first production-quality QRAEMON integration should not be considered complete until:

1. A robot can be connected through an adapter without modifying the QRAEMON cognitive core.
2. QRAEMON can consume canonical state and environment context.
3. QRAEMON can produce validated task/skill actions.
4. Existing low-level control remains functional if QRAEMON is stopped.
5. Safety systems can override QRAEMON.

6. QRAEMON handles stale, missing and contradictory inputs.
7. Actions can be cancelled deterministically.
8. All decisions and outcomes are traceable.
9. The system can operate in simulation.
10. The embedded version can boot and operate without Internet access.
11. Memory can be bounded, consolidated and selectively retrieved.
12. Memory-enabled performance can be compared against a no-memory baseline.

25 Recommended Immediate Engineering Backlog

1. Freeze the canonical QRAEMON/robot architecture.
2. Define the canonical robot-state schema.
3. Define the canonical action/skill schema.
4. Define the capability model.
5. Define action lifecycle and cancellation semantics.
6. Define safety-gateway contract.
7. Implement the adapter SDK.
8. Build a simulated robot adapter.
9. Implement the first memory API.
10. Implement context construction.
11. Implement a minimal planner.
12. Implement experience recording and outcome evaluation.
13. Build a no-memory baseline.
14. Build a memory-enabled experimental system.
15. Establish latency and reliability benchmarks.
16. Only then select the first physical robot and embedded compute platform.

26 Final Architectural Position

The intended QRAEMON platform is best understood as a **cognitive architecture with a stable embodiment interface**.

The two products are therefore not two different brains:

$$\text{QRAEMON Cognitive Core} + \begin{cases} \text{Software Deployment} \\ \text{Embedded Deployment} \end{cases}$$

The software product proves and distributes the cognitive architecture.

The embedded product packages that same architecture into an autonomous onboard computer for robots that must operate locally, including environments where external connectivity is unavailable, undesirable, or operationally inappropriate.

The strategic platform thesis is:

$$\boxed{\text{One cognitive architecture} \longrightarrow \text{many robot embodiments}}$$

with:

$$\boxed{\text{Robot-specific engineering isolated in adapters and embodiment interfaces}}$$

The long-term objective is therefore not to build a different AI system for every robot. It is to build a persistent cognitive layer whose core mechanisms remain stable while the body, sensors, actuators and low-level control systems can change.